

CLAM: Continuous Latent Action Models for Robot Learning from Unlabeled Demonstrations

Anthony Liang^{1,*}, Pavel Czempin^{1,*}, Matthew M. Hong¹, Yutai Zhou¹, Jingzhen Wang¹,
Erdem Bıyık^{1,2,†}, Stephen Tu^{1,2,†}
{anthony.liang,czempin}@usc.edu

Abstract—Learning robot control policies from demonstrations typically requires action-labeled expert data, which is expensive to collect through teleoperation. We study a more practical setting in which expert demonstrations are available only as observation sequences without action labels, and only task-agnostic play data contains actions. We introduce continuous latent action models (CLAM), a framework that infers continuous latent actions between consecutive observations using self-supervised dynamics prediction. To ground these latent actions into executable motor commands, CLAM jointly trains an action decoder using a small amount of task-agnostic play data. We show that continuous latent actions combined with this joint training are essential for high-dimensional continuous control. Across DMControl locomotion, MetaWorld manipulation, and real-world WidowX robot tasks, CLAM improves average task success rates by 2–3× over prior latent-action baselines and approaches behavior cloning trained with privileged expert action labels. Our results demonstrate that effective robot policies can be learned from unlabeled demonstrations and deployed on real hardware without collecting expert action-labeled data. Videos and code are available at clamrobot.github.io.

I. INTRODUCTION

Recent efforts in robotics attempted to tackle the collection of large-scale robot datasets [1–3]. However, data collection ultimately requires manual teleoperation, which is expensive and time-consuming. Furthermore, fine-tuning models in situ via teleoperation to address the diversity of real-world environments and tasks is challenging for laypeople.

A promising alternative is training on human video demonstrations. These are easier to collect than teleoperation data [4] and thus easier to scale to large datasets [5, 6]. However, utilizing human videos to train robot policies has two main challenges. First, standard imitation learning (IL) requires demonstrations from the robot’s own embodiment and viewpoint. Recent work addresses this by converting human videos to robot-compatible observations using hand tracking and inpainting [7, 8]. However, even with converted video, IL still requires ground-truth robot control signals, which these conversion methods do not provide. This is precisely the problem we focus on in this paper: learning

from expert demonstrations captured as robot video, but without action labels.

Actionless robot video is abundant in practice: human demonstrations can be converted to robot viewpoints [7, 8], internet video captures behaviors at scale [5, 6], and existing robot datasets might contain observation logs without compatible action formats. We introduce *continuous latent action models* (CLAM), a framework for learning robot control policies from observation-only robot demonstrations.

CLAM trains a latent inverse dynamics model to infer a continuous latent action between consecutive observations, supervised only by a forward dynamics model that reconstructs the next observation from the current observation and the latent action. This self-supervised loop requires no action labels which simplifies training on large-scale data. To make these latent actions executable on a real robot, CLAM jointly trains an action decoder that maps latent actions to motor commands, using only a small amount of task-agnostic play data. Joint training is critical: it regularizes the latent space so that it remains decodable, which we show is necessary for continuous control. Once trained, the latent inverse dynamics model annotates unlabeled expert demonstrations with pseudo-actions, and a policy is trained via standard IL on the resulting dataset.

However, realizing this pipeline for physical robots requires representations that map seamlessly to continuous robot control signals. In this work, we identify several key architectural improvements that greatly improve the paradigm of learning from observation-only data in continuous control robotics settings. Our contributions are threefold:

- 1) We demonstrate that latent action relabeling becomes practical for continuous robot control when using continuous latent spaces and joint grounding.
- 2) We demonstrate that task-agnostic play data is sufficient for grounding latent actions, dramatically reducing the cost of data collection for new tasks.
- 3) We deploy and evaluate CLAM on a physical WidowX robot across four manipulation tasks, showing it is competitive with privileged expert action labels without ever collecting action-labeled expert demonstrations.

II. RELATED WORK

Imitation from Observation. Learning robot policies from sequences of observations without access to actions is often referred to as *imitation from observation* (IfO) [9].

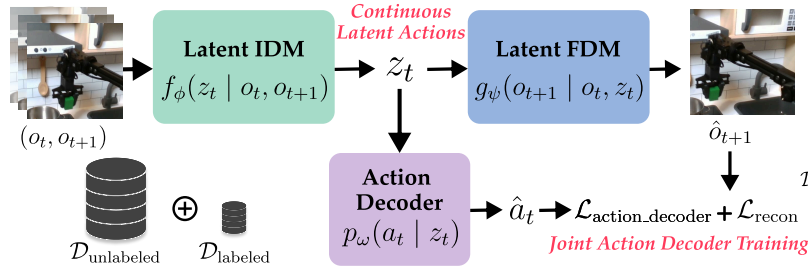
*Equal contribution

†Equal advising

¹Thomas Lord Department of Computer Science, University of Southern California

²Ming Hsieh Department of Electrical and Computer Engineering, University of Southern California

Stage 1: CLAM Pretraining



Stage 2: Latent Action Policy Training

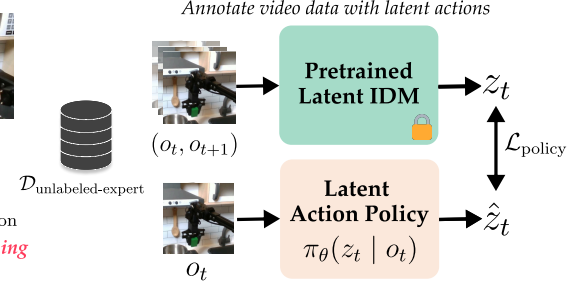


Fig. 1: **Overview of CLAM.** CLAM consists of a **latent inverse dynamics model**, f_ϕ , which infers the latent action between consecutive observations and **latent forward dynamics model**, g_ψ , which predicts the future observation conditioned on the latent action and observation history. CLAM learns a latent action space through the self-supervised objective of future observation reconstruction. Unlike prior work, CLAM produces *continuous latent actions*. To ensure the learned latent space is amenable to decoding to real-world actions, CLAM *jointly* trains the action decoder and the latent action model.

Unlike standard IL, demonstrations may differ in embodiment, environment, or viewpoint, and do not include action labels. Existing IFO approaches either learn policies directly from observation sequences by aligning trajectories [9–11], or predict future observations and map them to robot controls using learned or off-the-shelf perception models [12, 13]. However, these methods often depend on online rollouts or off-the-shelf vision models, which limit their applicability. In contrast, CLAM is trained fully offline, without rollouts or off-the-shelf models.

Supervised Learning of Inverse Dynamics Models.

With access to some action-labeled data, inverse dynamics models (IDMs) can be trained in a supervised fashion. VPT [14] and Seer [15], for example, use an IDM to label large unlabeled datasets, while UniPi [16] finetunes a video prediction model and applies an IDM to infer robot actions from synthetic data. A key limitation of such methods is that the IDM must predict actions in the *real* action space. Consequently, these methods are bottlenecked by the number of labeled demonstrations for task-specific fine-tuning or the budget required to collect them.

Latent Action Models. To bypass the dependency on labeled data, several works learn IDMs in a *latent* action space using only observation data [17–24]. These works apply a variety of subtle architectural differences, which we compare in a real-world setup. Works such as LAPA [21] and DynaMo [20] use latent action learning for pretraining to achieve better representation learning performance [20, 21, 25]. However, unlike CLAM, since the latent action model (LAM) is limited to pretraining, each new task still requires collecting labeled expert data to train a policy. Works such as LAPO [18] and Genie [19] combine VPT-style methods with latent action models, using the IDM to annotate unlabeled trajectories directly. While promising, they focus on video game domains with small discrete action spaces. These methods use vector quantization in the latent action space; recent works point toward the benefits of using continuous latent action spaces [26, 27]. We perform a detailed analysis in the real-world robotics domain, validating key architectural choices that make latent action modeling practical in fully

continuous, high-dimensional control settings.

A slew of concurrent works study the applicability of latent action learning to in-the-wild data [28], as a component of large behavior models [29], and world models [30, 31], highlighting the importance of this line of research.

LAOM [26] found that supervision with a small fraction of labeled actions improves the applicability of a learned latent action space to downstream tasks under distractors. Unlike LAOM, which used a single linear layer for supervision and discarded this layer at test time, our ground-truth action-supervision comes from the same jointly trained action decoder that is later used at inference time. Additionally, we focus on simulated and real-world robot manipulation tasks whereas Nikulin et al. [26] focus on invariance to distractors in simulated locomotion.

III. PROBLEM SETTING

We consider a practical setting in which a user wants to teach a robot a new task without collecting action-labeled expert demonstrations. The data regime involves three datasets with complementary roles. The bulk of the training data comes from a large corpus of robot video collected across diverse environments and tasks, without action labels. We denote this pool $\mathcal{D}_{\text{unlabeled}}$. Because no teleoperation or action logging is required, this type of data is cheap to scale.

Separately, we assume access to a small dataset of action-labeled robot transitions, $\mathcal{D}_{\text{labeled}}$. In our experiments, this consists largely of undirected, task-agnostic play data, which requires minimal expertise from the operator and no task-specific planning, making it substantially cheaper to collect than expert demonstrations [32]. $\mathcal{D}_{\text{labeled}}$ is used solely to ground latent actions into executable motor commands. Additionally, in our experiments, a small amount of task-specific demonstrations supplement the labeled data, but we assume they cannot be disambiguated from the play data at training time. Furthermore, we validate the effectiveness of CLAM on purely sub-optimal labeled data in our ablation studies.

The user’s only contribution is a small set of task-specific demonstrations, $\mathcal{D}_{\text{unlabeled-expert}}$, collected by teleoperating the robot and performing the target task themselves. These

Algorithm 1 CLAM w/ Joint Action Decoder Training

```
1: Input:  $\mathcal{D}_{\text{unlabeled}}, \mathcal{D}_{\text{labeled}}, \mathcal{D}_{\text{unlabeled\_expert}}, \text{IDM } f_\phi, \text{FDM } g_\psi, \text{Action Decoder } p_\omega, \text{Latent A. Policy } \pi_\theta$   
    $N_C$ : number of CLAM update steps  
    $N_P$ : number of policy updates steps  
    $K$ : train action decoder every  
   # Stage 1: Train CLAM and Action Decoder  
2: for iter = 1 in range( $N_C$ ) do  
3:   Update  $f_\phi$  and  $g_\psi$  with  $\mathcal{L}_{\text{recon}}$  on  $\mathcal{D}_{\text{unlabeled}}$   
4:   if iter %  $K == 0$  then  
5:     Update  $p_\omega$  and  $f_\phi$  with  $\mathcal{L}_{\text{action\_decoder}}$  on  $\mathcal{D}_{\text{labeled}}$   
   # Stage 2: Train Latent Action Policy  
6: Annotate  $\mathcal{D}_{\text{unlabeled\_expert}}$  with IDM  $f_\phi$   
7: for iter = 1 in range( $N_P$ ) do  
8:   Update  $\pi_\theta$  with  $\mathcal{L}_\pi$  on annotated  $\mathcal{D}_{\text{unlabeled\_expert}}$ 
```

demonstrations are expert with respect to the task but contain no action labels and are used to train the final policy.

To isolate the challenge of action inference from the orthogonal challenge of cross-embodiment transfer, we use robot demonstrations with action labels withheld as our unlabeled data. Additionally, all methods, including CLAM and baselines, have access to the same $\mathcal{D}_{\text{unlabeled}}, \mathcal{D}_{\text{labeled}}$, and $\mathcal{D}_{\text{unlabeled_expert}}$, however, some baselines might be bottlenecked by labeled demonstration requirements and thus cannot make use of some of the data. In this work, we focus on the single-task, single-robot-embodiment setting.

IV. CONTINUOUS LATENT ACTION MODELS

We introduce *continuous latent action models* (CLAM), shown in Figure 1, a scalable approach for training continuous control policies from unlabeled observation data. CLAM consists of two stages. In **Stage 1** (Section IV-A), we train a latent action model (LAM) for relabeling observation-only data. We then use this LAM in **Stage 2** (Section IV-B) to train a latent action policy.

A. Latent Action Model Training

In **Stage 1**, we pretrain a Latent Action Model (LAM) that we later use to annotate trajectories with pseudo-action labels. A LAM consists of two components: a *forward dynamics model* (FDM), which predicts the transition dynamics of the environment, and an *inverse dynamics model* (IDM), which inverts this process by inferring the action performed between two subsequent observations.

Algorithm 2 Inference Time Rollout

```
1: Input: Action Decoder  $p_\omega, \text{Latent Policy } \pi_\theta$   
2:  $o_t = \text{env.reset}()$   
3: while not done do  
4:    $z_t = \pi_\theta(\cdot | o_t)$  ▷ infer latent action  
5:    $a_t = p_\omega(\cdot | z_t)$  ▷ decode latent action  
6:    $o_t, \text{done} = \text{env.step}(a_t)$ 
```

Since we train these models without any action labels, we train a **latent IDM**, $f_\phi(z_t | o_t, o_{t+1})$, which predicts an

unobserved *latent* action z_t between two consecutive observations. This z_t encodes transition-specific state-dependent information needed to reconstruct the next observation. Thus it is a low-level action code and not a high-level skill label. To provide a training signal for the latent action, we jointly train a **latent FDM**, $g_\psi(o_{t+1} | o_t, z_t)$, to infer the next observation conditioned on the current observation and latent action. Since observations are *partial* and do not capture the full environment state, in practice, we provide the LAM with additional H steps of context making it easier to infer the underlying state and predict a more accurate latent action, i.e., $f_\phi(z_t | o_{t-H}, \dots, o_t, o_{t+1})$ and $g_\psi(o_{t+1} | o_{t-H}, \dots, o_t, z_t)$.

As shown in Figure 1, the training signal comes from future observation reconstruction, i.e. $\mathcal{L}_{\text{recon}} = \text{MSE}(\hat{o}_{t+1}, o_{t+1})$ where $\hat{o}_{t+1}$ is the FDM prediction. Our encoder/decoder architecture induces an information bottleneck, ensuring a meaningful, compact action representation rather than shortcut solutions. While prior works [18, 19] discretize latent actions from the IDM using Vector Quantization (VQ) [33], our experiments show that this fails in robotics tasks where actions are inherently continuous. We solve this shortcoming by replacing the VQ-based discretized action space with a learned continuous actions.

Latent Action Decoder. At test time, the learned latent actions cannot be directly executed in the environment. Consequently, we learn a **latent action decoder**, $p_\omega(a_t | z_t)$ using $\mathcal{D}_{\text{labeled}}$ to ground the learned latent actions to executable environment actions. To ensure z_t remains a low-level latent action, the action decoder is intentionally not conditioned on observations. Some prior work [18] trains the action decoder independently from the latent action model. This is reasonable for environments with discrete action spaces, where it is possible to learn the mapping from a discrete set of codes to the environment actions. Recent work [26] points toward the benefit of providing additional supervision by jointly training with labeled data. We find this holds in real-world experiments, so we adopt a joint training approach for our LAM training. We find that this allows for effective decoding to real-world actions.

Importantly, we do not make any assumptions about how $\mathcal{D}_{\text{labeled}}$ is collected. Furthermore, we demonstrate that $\mathcal{D}_{\text{labeled}}$ can come from any behavioral policy, even a random policy or task-agnostic play data, allowing CLAM to work even without access to expert teleoperated data. During LAM pretraining, we alternate between gradient updates on batches of unlabeled data for training the LAM and batches of labeled data for training the action decoder. The final training objective for CLAM is $\mathcal{L}_{\text{CLAM}} = \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{action-decoder}}$ where $\mathcal{L}_{\text{action-decoder}} = \text{MSE}(\hat{a}_t, a_t)$ and β is a hyperparameter that balances the reconstruction and action decoder losses.

B. Latent Action Policy Training

During **Stage 2**, we use the latent IDM from our pretrained CLAM to annotate $\mathcal{D}_{\text{unlabeled_expert}}$ with latent actions. The latent FDM only provides the learning signal for training the latent IDM and is discarded at this point. We ap-

	HALFCHEETAH	HOPPER	ASSEMBLY	BIN PICKING	PEG INSERT	SHELF PLACE	Average
BC-AL	0.22 $\pm$ 0.05	0.35 $\pm$ 0.04	0.34 $\pm$ 0.05	0.27 $\pm$ 0.12	0.29 $\pm$ 0.07	0.00 $\pm$ 0.00	0.24
LAPO	0.12 $\pm$ 0.05	0.24 $\pm$ 0.03	0.15 $\pm$ 0.04	0.02 $\pm$ 0.03	0.17 $\pm$ 0.04	0.06 $\pm$ 0.08	0.13
LAPA	0.22 $\pm$ 0.05	0.30 $\pm$ 0.06	0.24 $\pm$ 0.07	0.15 $\pm$ 0.01	0.25 $\pm$ 0.02	0.12 $\pm$ 0.02	0.21
DynaMo	0.18 $\pm$ 0.03	0.22 $\pm$ 0.02	0.10 $\pm$ 0.03	0.06 $\pm$ 0.03	0.12 $\pm$ 0.04	0.08 $\pm$ 0.02	0.13
VPT	0.32 $\pm$ 0.04	0.41 $\pm$ 0.03	0.40 $\pm$ 0.08	0.05 $\pm$ 0.02	0.49 $\pm$ 0.06	0.02 $\pm$ 0.00	0.28
MLP-CLAM*	0.64 $\pm$ 0.05	0.64 $\pm$ 0.03	0.53 $\pm$ 0.04	0.68 $\pm$ 0.05	0.58 $\pm$ 0.04	0.72 $\pm$ 0.04	0.63
Transformer-CLAM*	0.72 $\pm$ 0.04	0.81 $\pm$ 0.05	0.91 $\pm$ 0.03	0.82 $\pm$ 0.03	0.79 $\pm$ 0.07	0.93 $\pm$ 0.02	0.83
BC-Expert	0.68 $\pm$ 0.02	0.76 $\pm$ 0.04	1.00 $\pm$ 0.00	0.94 $\pm$ 0.05	0.91 $\pm$ 0.03	0.93 $\pm$ 0.00	0.87

TABLE I: **MetaWorld State-Based Results.** We report normalized returns for DMControl tasks and average task success rates for MetaWorld tasks. **Maroon** highlights the best method in each environment, excluding **BC-Expert**, which is trained with expert labeled data. Our methods, denoted by an asterisk (*), outperform all baselines across tasks.

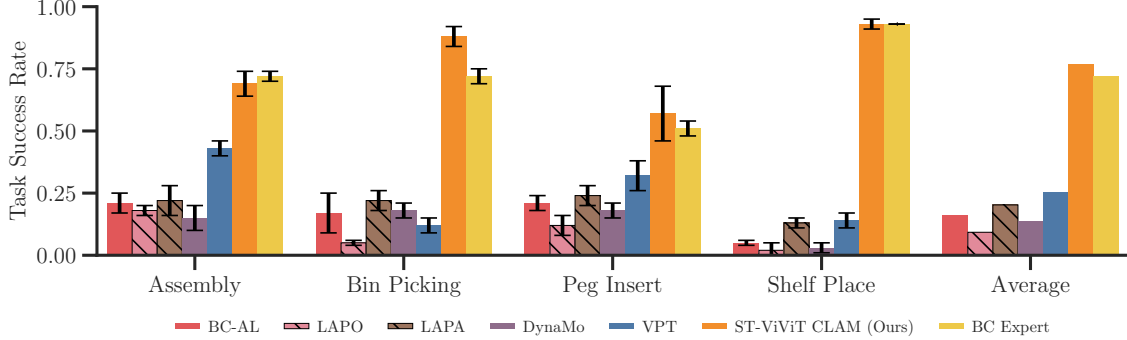


Fig. 2: **MetaWorld Image-Based Experiments.** We average task success rates over 50 evaluation rollouts across 3 random seeds using image-based inputs. We train all methods with the same amount of action-labeled data (100 trajectories) for fair comparison. Because $\mathcal{D}_{\text{labeled}}$ is not solely expert data, baselines struggle to learn a performant downstream policy, whereas **MLP-CLAM** and **Transformer-CLAM** achieve up to a 3 $\times$ improvement over the best baseline. Baselines using a *discrete* latent action space are indicated with hashed markers. We also report results for **BC-Expert**, trained on the same number of *labeled expert* trajectories, to illustrate the performance of a privileged reference trained with ground-truth actions.

ply the latent IDM to infer the latent action z_t between each consecutive observation (o_t, o_{t+1}) , i.e., $\mathcal{D}_{\text{relabelled-expert}} = \{(o_1^i, z_1^i, o_2^i, z_2^i, \dots, z_{T-1}^i, o_T^i) \mid \forall \tau^i \in \mathcal{D}_{\text{unlabeled-expert}}\}$.

Subsequently, we train a **latent action policy**, $\pi_\theta(z_t \mid o_t)$, using IL by optimizing $\mathcal{L}_\pi = \text{MSE}(\hat{z}_t, z_t)$ on batches of annotated data from $\mathcal{D}_{\text{relabelled-expert}}$. During inference time, detailed in Algorithm 2, our learned policy predicts the latent actions given an observation, which the action decoder will decode into an environment action.

Leveraging pretrained IDM image features for policy training. A side-effect of learning a LAM on image-based observations is that the IDM’s image encoders can be used as pretrained image features for the latent action policy (**Stage 2**). Viewed through this lens, training the LAM can be seen as a form of self-supervised representation learning. In DynaMo [20], it is shown that pretraining vision encoders using an IDM/FDM self-supervised loss improves the performance of downstream IL from expert-labeled demonstrations. We will show in Section V that similar positive transfer also occurs for CLAM when learning latent action policies.

V. EXPERIMENTAL SETUP

Environments and Datasets. We compare CLAM to several state-of-the-art baselines using both state- and image-based observations across tasks in DMControl [34], MetaWorld [35], and on a real WidowX robot arm outlined below.

Full details of the data split is provided in Table VI.

- **DMControl.** We evaluate on two DMControl locomotion tasks: HOPPER and HALFCHEETAH. We use trajectories from medium-expert split of the D4RL [36] benchmark for pretraining CLAM and subsample only suboptimal trajectories for $\mathcal{D}_{\text{labeled}}$.
- **MetaWorld.** We evaluate on four difficult MetaWorld tasks: ASSEMBLY, BIN PICKING, PEG INSERT, and SHELF PLACE. We train single-task RL agents and collect the replay buffer data for pretraining. We hold out a separate random-medium dataset similar to DMControl for $\mathcal{D}_{\text{labeled}}$.
- **WidowX Robot Arm.** We evaluate on four real world manipulation tasks (see bottom of Figure 3): REACH BLOCK, PUSH BUTTON, CLOSE MICROWAVE, and PUT OBJECT IN POT AND SLIDE POT which requires more than 150 timesteps at a 5 Hz control frequency. Our WidowX robot arm setup is shown in Figure 3 (Top). We collect a task-agnostic, play dataset of $\sim 50k$ transitions for $\mathcal{D}_{\text{unlabeled}}$ and use about 5k transitions for $\mathcal{D}_{\text{labeled}}$. We separately collect ~ 30 expert demonstrations per task without action labels for $\mathcal{D}_{\text{unlabeled-expert}}$.

All methods that predict environment actions use each benchmark’s native continuous action space. Ground-truth action spaces differ across benchmarks but not across tasks within a benchmark; see Table VII for full details.

Baselines. To ensure fair comparison, we reuse the same architectural components between CLAM and all baselines. These components are an IDM, an FDM, an action decoder/action head, and a transformer BC policy. Furthermore, all methods have access to same data when applicable. Some algorithms are unable to leverage certain subsets of the data, e.g. BC cannot use $\mathcal{D}_{\text{unlabeled}}$ since it requires action supervision. We compare CLAM to the following baselines:

- **BC-Action-Labeled (BC-AL):** Behavior cloning on the small $\mathcal{D}_{\text{labeled}}$, which is not fully expert data. Since BC needs action labels, it does not use $\mathcal{D}_{\text{unlabeled}}$.
- **VPT [14]:** The IDM is trained *only* on $\mathcal{D}_{\text{labeled}}$ via supervised learning. The IDM is used to label $\mathcal{D}_{\text{unlabeled-expert}}$ with environment actions and a BC policy is then trained on the annotated data.
- **LAPO [18]:** Latent action model with discrete, vector-quantized latent actions.
- **LAPA [21]:** After LAM pretraining, the final layer of the IDM is replaced with an action head and fine-tuned end-to-end on $\mathcal{D}_{\text{labeled}}$, which is *non-expert*.
- **DynaMo [20]:** Self-supervised learning on $\mathcal{D}_{\text{unlabeled}}$ to train a vision encoder using an IDM and FDM to predict latent embeddings of future frames. A BC policy is trained on the image embeddings produced by the pretrained vision encoder using $\mathcal{D}_{\text{labeled}}$. We include this baseline to compare what a pure image-feature pretraining approach can achieve in our problem setting, which does not use the LAM for relabeling.
- **MLP-, Transformer-, ST-ViViT-CLAM (Ours):** Continuous latent action model with different parameterizations of the latent IDM and FDM.
- **BC-Expert (BC-E):** Privileged BC on $\mathcal{D}_{\text{unlabeled-expert}}$ with *ground-truth* action labels available, which are not available to other methods.

CLAM Model Architectures: For image observations, we model after the Space-Time (ST) Transformer [37]. We first patchify a $64 \times 64 \times 3$ image with a patch size of 16 for a total of 16 patches. Each patch is embedded through a linear layer into the hidden dimension. The encoder consists of N_E layers of Space-Time (ST) Attention blocks. Each ST block consists of spatial attention followed by temporal attention and a feedforward layer with skip connection, LayerNorm, and dropout applied between each attention. The decoder ST block also applies a cross-attention with the latent actions generated by the encoder. We add an additional token in the sequence of patch embeddings as a CLS token for the whole image. From the CLS token for each timestep, we apply a linear layer to predict the latent actions.

Latent Policy Architecture: For our policy, we use a transformer decoder similar to the one used in ACT [38]. For external and over-the-shoulder RGB images, we use a pretrained ResNet to extract 7×7 feature maps and flatten them along the spatial dimension to create a sequence of d_v -dimensional tokens, where d_v is the output dimension of the ResNet. We use ResNet18 with $d_v = 512$. We feed as input to a causal transformer decoder a sequence of learnable

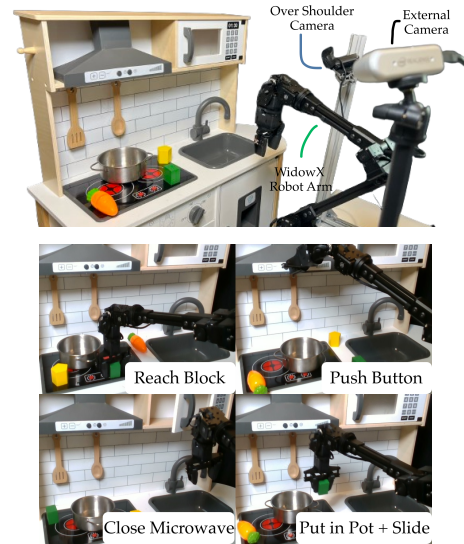


Fig. 3: **WidowX Robot Arm Setup and Evaluation Tasks.** (Top) WidowX robot arm in a toy kitchen setup [39]. We use an Intel Realsense D435 RGBD camera as a fixed external camera and a Logitech C920 webcam as an over-the-shoulder camera view. (Bottom) Four tabletop manipulation tasks used to evaluate the scalability of CLAM.

action tokens with dimension d . The flattened image feature map is the key and value, and we apply cross-attention between the image features and learnable tokens. We concatenate all modality tokens and add additional modality-specific embeddings and sinusoidal positional embeddings.

The input sequence to the transformer is a fixed position embedding, with dimensions $k \times 512$ where k is the chunk size and the keys and values are the combined image tokens from the stem. At each timestep, we predict a chunk of 5 actions corresponding to 1 second of execution.

VI. RESULTS

We aim to study the efficacy of CLAM as a general approach to learn from actionless data, evaluate its ability to train robot policies without access to labeled expert data, and analyze its design choices and limitations. We organize our experiments to answer the following:

- (Q1) How effective is CLAM at learning policies *without* action-labeled expert demonstrations?
- (Q2) How important is each component of CLAM: *continuous* latent actions and *jointly* training action decoder?
- (Q3) Can CLAM scale to efficiently learn capable robot policies for real-world scenarios?

FINDING 1: CLAM outperforms all baselines and nearly matches the performance of BC with expert data in state- and image-based experiments. Table I summarizes our results for state-based inputs on DMControl and Figure 2 for image-based results in MetaWorld. CLAM improves upon the best baseline VPT by more than $2\times$ average normalized return on the DMControl (*locomotion*) tasks and around $2-3\times$ success rate in MetaWorld (*manipulation*).

BC-AL using *action-labeled* data unsurprisingly does not perform well due to imitating suboptimal demonstrations. In

	ASSEMBLY	BIN PICK	PEG INSERT	SHELF PLACE
Disc., $\neg$ JT	0.15 $\pm$ 0.03	0.12 $\pm$ 0.02	0.18 $\pm$ 0.04	0.19 $\pm$ 0.03
Disc., JT	0.14 $\pm$ 0.04	0.14 $\pm$ 0.03	0.17 $\pm$ 0.03	0.16 $\pm$ 0.04
Cont., $\neg$ JT	0.28 $\pm$ 0.04	0.18 $\pm$ 0.03	0.21 $\pm$ 0.08	0.26 $\pm$ 0.06
Cont., JT	0.69 $\pm$ 0.05	0.82 $\pm$ 0.04	0.57 $\pm$ 0.11	0.88 $\pm$ 0.02

TABLE II: **CLAM Ablation Study.** Both continuous latent actions and joint training are necessary to improve task success. Using continuous actions (row 2 $\rightarrow$ row 3) yields about a $1.5\times$ improvement in success rates. Joint training (row 3 $\rightarrow$ row 4) further boosts performance by roughly $3\times$. JT refers to *joint action decoder training*.

several tasks, **Transformer-CLAM** achieves performance close to or even better than that of **BC-Expert** which uses the same amount of privileged *expert* action-labeled data. Although **BC-Expert** has access to privileged expert action labels, it is trained only on the limited expert dataset. CLAM, by contrast, uses the larger unlabeled corpus during LAM pretraining before fitting the downstream latent policy. Thus, when CLAM approaches or exceeds **BC-Expert**, we interpret the result as evidence that self-supervised pretraining can compensate for missing expert action labels by learning task-relevant representations and a smoother latent action space. In image-based experiments, this benefit is consistent with transfer from the pretrained IDM encoder (cf. Section IV-B). In state-based experiments, the latent-action bottleneck may act as a regularizer by filtering high-variance action targets. We leave a full causal attribution to targeted ablations, but the comparison shows that privileged expert labels are not always sufficient to overcome the data and representation advantages provided by CLAM pretraining.

All variants of CLAM outperform the best baseline **VPT** [14], highlighting the fact that latent action models scale with $|\mathcal{D}_{\text{unlabeled}}|$ while supervised IDMs only scale with $|\mathcal{D}_{\text{labeled}}|$. Since our problem setup assumes $|\mathcal{D}_{\text{labeled}}| \ll |\mathcal{D}_{\text{unlabeled}}|$, it is likely that **VPT** learns a suboptimal IDM, underscoring the benefit of *latent* action models which can leverage vast, unstructured observation data to learn latent actions in an unsupervised manner. CLAM outperforms state-of-the-art methods in our problem setting where only *play* data is available as action-labeled data, and expert data is actionless. In other data settings, the baselines will likely be competitive, and thus choosing the right method for learning is dependent on the specific data regime. We emphasize that our data regime enables scalable learning from easy-to-collect, cheap play data [40] avoiding the need for expensive task-specific data collection.

FINDING 2: Continuous latent actions and joint action decoder greatly improve performance in real-world robotics tasks. First, we corroborate the findings by Nikulin et al. [26] that continuous latent actions joint training benefit LAM performance. Unlike **VPT**, the other baselines (**LAPo**, **LAPA**, and **DynaMo**) make use of LAMs, as does our method. We find that we can outperform these methods, likely due to using continuous latent actions in conjunction with jointly training an action decoder. First, baselines that apply vector quantization [33] to discretize the latent

	BLOCK	BUTTON	MICROWAVE	SLIDE POT
BC-AL	0/10	0.5/10	1/10	0/10
LAPA	2/10	3/10	3/10	0/10
VPT	2.5/10	4/10	5/10	2/10
ST-CLAM	7/10	8.5/10	8/10	4/10
BC-Expert	7.5/10	8/10	7.5/10	2/10

TABLE III: **Real Robot Results.** **ST-CLAM** significantly outperforms baseline methods across all tasks. Each entry sums scores over 10 trials, where partial score or full score is awarded (0.5/1 point). Partial success is, block—reach/pick up; button—near/touch; microwave—touch/close; slide pot—reach/slide.

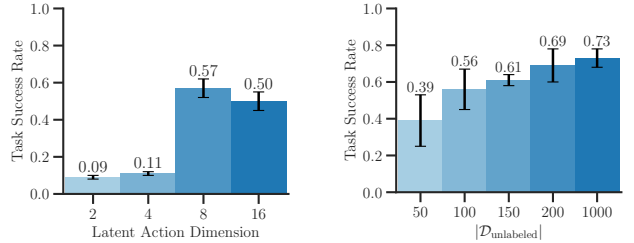


Fig. 4: **(Left)** Up until a latent dimension of 4, the learned latent action space fails to be useful for IL. However, a latent dimension of 8 has sufficient capacity, achieving 57% success rate on the *Assembly* task. **(Right)** CLAM scales with the amount of unlabeled demonstration data. The performance of the downstream policy improves as we annotate more trajectories using the pretrained CLAM.

actions, including **LAPo** and **LAPA**, perform poorly on continuous control tasks. In our image-based experiments, **ST-ViViT-CLAM** achieves an over $3\times$ improvement in task success rate at 76%, compared to **LAPo** and **LAPA**, which achieve 9% and 20%, respectively (cf. Figure 2). Prior works utilize VQ primarily to simplify the structure of the latent action space which is a reasonable choice for discrete action environments. We hypothesize that applying quantization to the latent space severely limits the expressivity of the latent actions for fine-grained manipulation tasks.

A potential issue with using an arbitrary continuous latent action space is grounding actions in the environment. In discrete settings, LAMs can recover an action space corresponding to a permutation of the ground-truth environment actions [19]. However, learning this mapping is more challenging for continuous actions. In Table II, we present additional experiments, ablating both the choice of action space (discrete vs. continuous) and joint training on MetaWorld tasks. Using discrete latent actions, equivalent to **LAPo** [18], results in an average of 16% task success compared to using continuous latent actions, which achieves 23%. This indicates that even without joint training, continuous actions improve performance. Furthermore, while joint training does not help much in the discrete latent action case, we see a substantial improvement when coupled with continuous latent actions, achieving 74% average success rate (over $3\times$ improvement).

Real WidowX Robot Arm Experiments. We evaluate

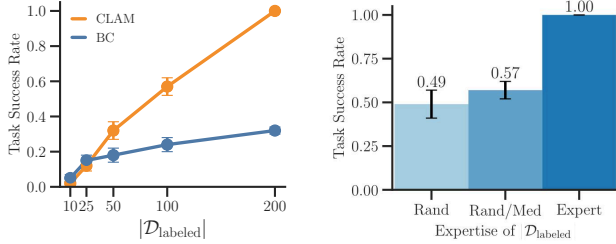


Fig. 5: **(Left)** We vary the number of labeled trajectories for training the action decoder. While BC performance struggles to learn from *non-expert* data, our method improves with more data. **(Right)** We also evaluate the robustness of CLAM to data of varying expertise. We learn a better policy than BC with the same amount of labeled random trajectories. With expert data, our method recovers an optimal policy.

the scalability of CLAM to more realistic applications on a physical WidowX robot arm shown in Figure 3 (Left). Even though $\mathcal{D}_{\text{labeled}}$ comprises of *task-agnostic*, *mixed expertise* *play* data, we demonstrate that CLAM is still able to learn to solve the tasks while baseline methods struggle to leverage this data to train a task-specific policy. As in the simulated experiments, we find that **VPT** is the closest baseline, but still struggles as it can only scale with the limited amount of labeled data, while **BC-AL** and **LAPA** are unable to learn from *play* data entirely. Remarkably, CLAM learns a policy to solve new tasks, without having explicitly collected action-labeled, expert demonstrations. This finding validates prior findings of benefits from action supervision during LAM training, as well as continuous latent action spaces in simulation settings [26] and smaller-scale experiments [27].

FINDING 3: CLAM successfully learns without ever accessing action-labeled expert data in both simulation and real robot experiments. We perform our main-line experiments (Figure 2) with action-labeled data that is not fully expert. **LAPA** circumvents the issues of learning a separate action-grounding model by directly fine-tuning the pretrained LAM with an uninitialized action prediction head on $\mathcal{D}_{\text{labeled}}$. However, if only partially-optimal or play data is available, the fine-tuned BC model struggles to learn a good policy, since it does not label the unlabeled data. In this regard, **LAPA** is similar to **BC-AL** with a better initialization as a result of the LAM pretraining, (20% vs. 16% success rate). Despite the action-labeled data not being fully optimal, CLAM is still able to achieve high success rates.

To further investigate this capability of our method, we compare data compositions of varying expertise levels. We filter our offline dataset for 100 trajectories below a predefined threshold return value and manually verify that the policy does not achieve the task and acts randomly. We show in Figure 5 (Right) that even with *random* policy data we can achieve similar performance as on the *random/medium* data regime. When *expert* data is available, we can recover a policy that always solves the task.

Latent action dimension directly affects the model’s expressivity. In Figure 4 (Left), we vary the latent action

dimension $|z| \in \{2, 4, 8, 16\}$ for the MetaWorld Assembly task. We find that setting $|z|$ to the true action dimension of 4 in this task is insufficient. Instead, a slight overparameterization is necessary to achieve a reasonable success rate. This suggests that even when our LAM is capable of learning a viable action space, it might fail at learning the most compact action representation possible. Further increasing $|z|$ to 16 does not yield any additional gains, likely because once the LAM is capable of learning a viable action space, there is no need for further representational capacity.

Latent action policy scales with $|\mathcal{D}_{\text{unlabeled-expert}}|$. In Figure 4 (Right) we demonstrate that CLAM’s performance on the Assembly task improves as we increase $|\mathcal{D}_{\text{unlabeled}}|$. This result suggests that we can improve robot policies without the need for expensive, manually collected expert teleoperated demonstrations. We note that the returns start diminishing after a certain number of trajectories, likely because the action decoder’s data remains unchanged.

Increasing $|\mathcal{D}_{\text{labeled}}|$ improves the action decoder accuracy and downstream performance. In Figure 5 (Left) we analyze the effect of varying $|\mathcal{D}_{\text{labeled}}|$ for training the action decoder. With only a handful of trajectories, it is difficult to ground the latent actions to the environment explaining the poor performance. As we increase $|\mathcal{D}_{\text{labeled}}|$, the learned action decoder becomes more accurate and better generalizes to new unseen states. Conversely, BC trained using the same amount of labeled *non-expert* data quickly plateaus in performance and fails to scale to the performance of CLAM, even with up to 100 labeled trajectories.

VII. CONCLUSION

We proposed CLAM, a scalable solution for learning continuous control tasks from unlabeled robot demonstration data. We show with real robot experiments that CLAM, using continuous latent actions and joint training, can learn policies for unseen tasks without requiring any expert teleoperated demonstrations. We hope CLAM further advances the scalable training of robot policies from action-less data, alleviating the need for expensive, manual data collection.

Generalizing to real-world videos. To train foundation policies from in-the-wild Internet scale video data, a number of additional challenges need to be addressed. In-the-wild internet videos might contain visual artifacts such as motion blur, low resolution, and compression artifacts, which might have to be addressed with existing computer vision techniques. Another challenge is the embodiment gap between human videos and the target robot embodiment.

Embodiment gap. The human-robot embodiment gap is directly addressable with video conversion methods such as Phantom [7] and Masquerade [8]. Alternatively, prior works such as CrossFormer [41] fine-tune action heads for different embodiments, but are limited to a fixed set of embodiments seen during training. For future work, we aim to apply the action-less learning capabilities of CLAM to the cross-embodiment problem, to allow learning for various robot embodiments and the human embodiment in a scalable way.

APPENDIX

Hyperparameter	Value
Num updates	500,000
Train action decoder every	2
Action Decoder batch size	128
Action Decoder loss weight	1
Action Decoder hidden dim	[1024, 1024, 1024]
Action Decoder embedding dim	512
Reconstruction loss weight	1
Latent action dim	16
Context len	2
Embedding dim	128

TABLE IV: CLAM Pretraining Hyperparameters

Hyperparameter	Value
Num encoder layers	3
Num decoder layers	3
Model dimension	256
Feedforward dimension	2048
Num attention head	4
Dropout	0.1
Pre norm	False
Feedforward activation	GeLU
Position Encoding	Learned

TABLE V: Transformer CLAM Model Hyperparameters

	MuJoCo	MetaWorld	Real Robot
$\mathcal{D}_{\text{unlabeled}}$	1000	1000	~500
$\mathcal{D}_{\text{labeled}}$	50	50	50
$\mathcal{D}_{\text{unlabeled_expert}}$	20	20	30/50

TABLE VI: Data Splits (# trajectories)

Domain	Action Dim
Hopper	3
HalfCheetah	6
MetaWorld (all tasks)	4
Real WidowX (all tasks)	7

TABLE VII: Ground-Truth Action Dimensions

REFERENCES

- [1] H. Xiong et al., “Robotube: Learning household manipulation from human videos with simulated twin environments,” in *Conference on Robot Learning*, 2023.
- [2] Open X-Embodiment Collaboration, “Open x-embodiment: Robotic learning datasets and rt-x models,” *arXiv preprint arXiv:2310.08864*, 2023.
- [3] K. Black et al., “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [4] M. Hong et al., *Hand me the data: Fast robot adaptation via hand path retrieval*, 2025.
- [5] K. Grauman et al., “Ego4d: Around the world in 3,000 hours of egocentric video,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 18 973–18 990.
- [6] D. Damen et al., “Rescaling egocentric vision: Collection, pipeline and challenges for epic-kitchens-100,” *International Journal of Computer Vision (IJCV)*, vol. 130, pp. 33–55, 2022.
- [7] M. Lepert, J. Fang, and J. Bohg, “Phantom: Training robots without robots using only human videos,” in *9th Annual Conference on Robot Learning*, 2025.
- [8] M. Lepert, J. Fang, and J. Bohg, “Masquerade: Learning from in-the-wild human videos using data-editing,” *arXiv preprint arXiv:2508.09976*, 2025.
- [9] Y. Liu et al., “Imitation from observation: Learning to imitate behaviors from raw video via context translation,” in *International Conference on Robotics and Automation*, 2018.
- [10] C. Yang et al., “Imitation learning from observations by minimizing inverse dynamics disagreement,” in *Advances in Neural Information Processing Systems*, 2019.
- [11] F. Liu et al., “State alignment-based imitation learning,” in *International Conference on Learning Representations*, 2020.
- [12] C. Wen et al., *Any-point trajectory modeling for policy learning*, 2023.
- [13] P.-C. Ko et al., “Learning to act from actionless videos through dense correspondences,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [14] B. Baker et al., “Video pretraining (vpt): Learning to act by watching unlabeled online videos,” in *Advances in Neural Information Processing Systems*, 2022.
- [15] Y. Tian et al., “Predictive inverse dynamics models are scalable learners for robotic manipulation,” *International Conference on Learning Representations*, 2024.
- [16] Y. Du et al., “Learning universal policies via text-guided video generation,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [17] W. Menapace et al., “Playable video generation,” in *Conference on Computer Vision and Pattern Recognition*, 2021.
- [18] D. Schmidt and M. Jiang, “Learning to act without actions,” in *International Conference on Learning Representations*, 2023.
- [19] J. Bruce et al., “Genie: Generative interactive environments,” in *International Conference on Machine Learning*, 2024.
- [20] Z. J. Cui et al., “Dynamo: In-domain dynamics pretraining for visuomotor control,” in *International Conference on Machine Learning*, 2024.
- [21] S. Ye et al., “Latent action pretraining from videos,” in *International Conference on Learning Representations*, 2024.
- [22] A. Klepach et al., “Object-centric latent action learning,” *arXiv preprint arXiv:2502.09680*, 2025.
- [23] Q. Bu et al., “Learning to act anywhere with task-centric latent actions,” *Robotics: Science and Systems*, 2025.
- [24] X. Chen et al., “Villa-x: Enhancing latent action modeling in vision-language-action models,” *arXiv preprint arXiv: 2507.23682*, 2025.
- [25] B. Tharwat et al., *Latent action pretraining through world modeling*, 2025.
- [26] A. Nikulin et al., “Latent action learning requires supervision in the presence of distractors,” in *Forty-second International Conference on Machine Learning*, 2025.
- [27] J. Yang et al., *Como: Learning continuous latent motion from internet videos for scalable robot learning*, 2025.
- [28] Q. Garrido et al., *Learning latent action world models in the wild*, 2026.
- [29] F. Lin et al., *A systematic study of data modalities and strategies for co-training large behavior models for robot manipulation*, 2026.
- [30] S. Ye et al., *World action models are zero-shot policies*, 2026.
- [31] S. Gao et al., “Dreamdojo: A generalist robot world model from large-scale human videos,” *arXiv preprint arXiv:2602.06949*, 2026.
- [32] C. Lynch et al., “Learning latent plans from play,” in *Conference on Robot Learning*, 2020.
- [33] A. Van Den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural discrete representation learning,” *Advances in Neural Information Processing Systems*, 2017.
- [34] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *International Conference on Intelligent Robots and Systems*, 2012.
- [35] T. Yu et al., “Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conference on Robot Learning*, 2020.
- [36] J. Fu et al., *D4rl: Datasets for deep data-driven reinforcement learning*, 2020.
- [37] G. Bertasius, H. Wang, and L. Torresani, “Is space-time attention all you need for video understanding?” in *International Conference on Machine Learning*, 2021.
- [38] T. Z. Zhao et al., “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [39] H. R. Walke et al., “Bridgedata v2: A dataset for robot learning at scale,” in *Conference on Robot Learning*, 2023.
- [40] O. Mees et al., “Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” in *IEEE Robotics and Automation Letters*, 2022.
- [41] R. Doshi et al., “Scaling cross-embodied learning: One policy for manipulation, navigation, locomotion and aviation,” in *Conference on Robot Learning*, 2024.